

Rockchip RK3506 RT-Thread SDK 快速入门

文档标识: RK-JC-YF-A34

发布版本: V1.1.0

日期: 2024-11-29

文件密级: ☐绝密 ☐秘密 ☐内部资料 ☒公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2024 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址: 福建省福州市铜盘路软件园A区18号

网址: www.rock-chips.com

客户服务电话: +86-4007-700-590

客户服务传真: +86-591-83951833

客户服务邮箱: fae@rock-chips.com

前言

概述

本文主要描述了RK3506 SMP RT-Thread SDK的基本使用方法，旨在帮助开发者快速了解并使用RK3506 SDK开发包。

各芯片系统支持状态

芯片名称	RT-Thread 版本
RK3506	4.1

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

日期	版本	作者	修改说明
2024-10-22	V1.0.0	Jiahang Zheng	初始版本
2024-11-29	V1.1.0	Jiahang Zheng	添加文件系统和 OTA 升级

目录

Rockchip RK3506 RT-Thread SDK 快速入门

1. 开发环境搭建
 - 1.1 准备开发环境
 - 1.2 安装库和工具集
 - 1.2.1 检查和升级主机的 `scons` 版本
 - 1.2.2 检查和升级主机的 `python` 版本
 - 1.2.3 检查和升级主机的 `make` 版本
2. 软件开发指南
 - 2.1 开发向导
 - 2.2 芯片资料
3. 硬件开发指南
4. 目录结构
5. SDK交叉编译工具链介绍
 - 5.1 U-Boot 编译工具链
 - 5.2 RT-Thread 编译工具链
6. SDK编译说明
 - 6.1 SDK编译命令查看
 - 6.2 SDK板级配置
 - 6.3 配置文件
 - 6.3.1 统一编译配置文件
 - 6.4 RT-Thread 固件打包配置文件
 - 6.5 分区表配置文件
 - 6.6 统一编译命令
 - 6.7 单独编译命令
 - 6.7.1 U-Boot 编译命令
 - 6.7.2 RT-Thread 编译命令
 - 6.7.3 `root.img` 打包
 - 6.7.4 RK HAL 编译命令
7. RT-Thread 资源配置
 - 7.1 RT-Thread 配置文件
 - 7.1.1 板级相关文件
 - 7.1.2 编译相关文件
 - 7.2 RT-Thread 内存资源
 - 7.2.1 AP 运行内存配置
 - 7.3 编译链接脚本介绍
 - 7.3.1 概述
 - 7.3.1.1 栈大小分配
 - 7.3.1.2 堆大小分配
8. OTA 升级
 - 8.1 固件格式
 - 8.2 配置说明：
 - 8.2.1 配置 `parameter.txt`
 - 8.2.2 UBoot 配置
 - 8.2.3 RT-Thread 配置
 - 8.2.4 版本号修改
 - 8.3 A/B固件启动信息
 - 8.4 OTA触发方式
 - 8.4.1 默认触发方式
 - 8.4.2 其它触发方式
9. 刷机说明
 - 9.1 Windows 刷机说明
 - 9.2 Linux 刷机说明
10. 运行调试

10.1 系统启动

10.2 串口调试

1. 开发环境搭建

1.1 准备开发环境

我们推荐使用 Ubuntu 22.04 或更高版本的系统进行编译。除了系统要求外，还有其他软硬件方面的要求。

硬件要求：64 位系统，硬盘空间大于 40G。如果您进行多个构建，将需要更大的硬盘空间。

软件要求：Ubuntu 22.04 或更高版本系统。

1.2 安装库和工具集

使用命令行进行设备开发时，可以通过以下步骤安装编译SDK需要的库和工具。

使用如下apt-get命令安装后续操作所需的库和工具：

```
sudo apt-get update && sudo apt-get install git ssh make gcc python-is-python3 python2
```

说明：

安装命令适用于Ubuntu22.04，其他版本请根据安装包名称采用对应的安装命令，若编译遇到报错，可以视报错信息，安装对应的软件包。其中：

- python要求安装python 3.6及以上版本，此处以python 3.6为例。
- make要求安装 make 4.0及以上版本，此处以 make 4.2为例。
- scons要求安装 v4.0.1 及以上版本。

1.2.1 检查和升级主机的 **scons** 版本

检查和升级主机的 **scons** 版本方法如下：

- 检查主机 **scons** 版本

```
$ scons --version
SCons by Steven Knight et al.:
    SCons: v4.0.1.c289977f8b34786ab6c334311e232886da7e8df1, 2020-07-17
01:50:03, by bdbaddog on ProDog2020
    SCons path: ['/usr/lib/python3/dist-packages/SCons']
Copyright (c) 2001 - 2020 The SCons Foundation
```

如果不满足scons>=4.0.1版本的要求，可通过如下方式升级：

- 升级 SCons: v4.0.1 新版本

```
wget https://sourceforge.net/projects/scons/files/scons/4.0.1/scons-4.0.1.zip
unzip scons-4.0.1.zip
cd SCons-4.0.1/
sudo python setup.py install
```

1.2.2 检查和升级主机的 **python** 版本

检查和升级主机的 **python** 版本方法如下：

- 检查主机 **python** 版本

```
$ python3 --version
Python 3.10.6
```

如果不满足 **python**>=3.6版本的要求， 可通过如下方式升级：

- 升级 **python** 3.6.15 新版本

```
PYTHON3_VER=3.6.15
echo "wget
https://www.python.org/ftp/python/${PYTHON3_VER}/Python-${PYTHON3_VER}.tgz"
echo "tar xf Python-${PYTHON3_VER}.tgz"
echo "cd Python-${PYTHON3_VER}"
echo "sudo apt-get install libsqlite3-dev"
echo "./configure --enable-optimizations"
echo "sudo make install -j8"
```

1.2.3 检查和升级主机的 **make** 版本

检查和升级主机的 **make** 版本方法如下：

- 检查主机 **make** 版本

```
$ make -v
GNU Make 4.2
Built for x86_64-pc-linux-gnu
```

- 升级 **make** 4.2 新版本

```
$ sudo apt update && sudo apt install -y autoconf autopoint

git clone https://gitee.com/mirrors/make.git
cd make
git checkout 4.2
git am $BUILDDROOT_DIR/package/make/*.patch
autoreconf -f -i
./configure
make make -j8
sudo install -m 0755 make /usr/bin/make
```

2. 软件开发指南

2.1 开发向导

瑞芯微提供的 RT-Thread 有 SMP 和 AMP 两种系统，AMP 下 RT-Thread 只占用一个核，其余核可以运行其他系统如 Linux，本文只针对 SMP RT-Thread。为帮助开发工程师更快上手熟悉 RT-Thread SDK 的开发调试工作，各个模块驱动文档随 SDK 发布，可在 `docs/cn/Common` 目录下获取，并会不断完善更新。

2.2 芯片资料

为帮助开发工程师更快上手熟悉 RK3506 的开发调试工作，随 SDK 发布《Rockchip RK3506 Datasheet V0.1-20240516.pdf》芯片手册。可在 `docs/cn/RK3506/Datasheet` 目录下获取，并会不断完善更新。

3. 硬件开发指南

硬件相关开发可以参考用户使用指南，在工程目录：

RK3506 硬件设计指南：

```
<SDK>/docs/cn/RK3506/Hardware/
```

4. 目录结构

以下是SDK主要目录对应的说明：

```
.
├── app
│   └── rkadk
├── build.sh           # SDK 统一编译脚本
├── device
│   └── rockchip       # SDK 统一编译配置及工具目录
├── docs               # SDK 相关文档
├── external           # 存放第三方相关仓库
├── hal
│   ├── doc           # HAL 相关文档
│   ├── html          # HAL doxygen 生成驱动文档
│   ├── lib           # HAL 驱动代码
│   ├── project       # HAL 工程目录
│   ├── test          # HAL 测试代码
│   └── tools          # HAL 相关工具
├── prebuilts          # SDK 编译工具链(包括编译U-Boot、HAL编译工具链)
├── rkbin              # SDK 相关Binary文件(包括Loader、Trust等打包用基础bin文件)
└── rockdev            # SDK 生成镜像目录
```

```

├─ rtos          # RT-Thread相关代码
├─ tools
│   └─ linux    # Linux 平台烧录工具
│   └─ mac      # Mac 平台烧录工具
│   └─ standalone # 烧录工具配置参数
│   └─ windows  # Windows 平台烧录工具及驱动
└─ u-boot       # U-boot相关代码

```

5. SDK交叉编译工具链介绍

鉴于 Rockchip RT-Thread SDK 目前只在 Linux PC 环境下编译，我们也仅提供了 Linux 下的交叉编译工具链。U-Boot 和 RT-Thread 使用的编译工具链不同。

5.1 U-Boot 编译工具链

SDK prebuilts 目录预置交叉编译目前用于U-Boot 编译，如下：

目录	说明
prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi	gcc arm 10.3.1 32位工具链

5.2 RT-Thread 编译工具链

目录	说明
prebuilts/gcc/linux-x86/arm/gcc-arm-none-eabi-10-2020-q4-major-x86_64-linux	gcc arm 32位工具链

6. SDK编译说明

SDK可通过 `make` 或 `./build.sh` 加目标参数进行相关功能的配置和编译。

具体参考 `device/rockchip/common/README.md` 编译说明。

为了确保SDK每次更新都能顺利进行，建议在更新前清理之前的编译产物。这样做可以避免潜在的兼容性问题或编译错误，因为旧的编译产物可能不适用于新版本的SDK。要清理这些编译产物，可以直接运行命令 `./build.sh cleanall`。

一键编译

芯片	类型	参考机型	一键编译
RK3506	开发板	RK3506G EVB1	<code>./build.sh lunch:rockchip_rk3506_g_evb1_smp_defconfig && ./build.sh</code>

编译成功后在 `<SDK>/rockdev/` 下有如下固件：

```
.
├── amp.img
├── MiniLoaderAll.bin -> ../../u-boot/rk3506_spl_loader_v1.02.110.bin
├── parameter.txt -> ../../device/rockchip/.chips/rk3506/parameter-128M-smp.txt
├── root.img
├── uboot.img -> ../../u-boot/uboot.img
└── update.img -> ../update/Image/update.img

0 directories, 6 files
```

注意：

- **U-boot**编译：其中需要制指定工具链.
- 比如SDK内置32位工具链：
`export CROSS_COMPILE=/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi-`
- **RT-Thread** 编译：
`export RTT_EXEC_PATH=/prebuilts/gcc/linux-x86/arm/gcc-arm-none-eabi-10-2020-q4-major-x86_64-linux/bin/`

6.1 SDK编译命令查看

`make help` , 例如：

```
$ make help
menuconfig          - interactive curses-based configurator
oldconfig           - resolve any unresolved symbols in .config
synconfig           - Same as oldconfig, but quietly, additionally update
deps
olddefconfig        - Same as synconfig but sets new symbols to their
default value
savedefconfig       - Save current config to RK_DEFCONFIG (minimal config)
...
```

`make`实际运行是 `./build.sh`

即也可运行 `./build.sh <target>` 来编译相关功能，具体可通过 `./build.sh help` 查看具体编译命令。

6.2 SDK板级配置

进入工程 `<SDK>/device/rockchip/rk3506` 目录：

板级配置	说明
rockchip_rk3506_g_evb1_smp_defconfig	适用于使用RT-Thread SMP方案的RK3506G EVB1 板子 CPU0~CPU2: RT_Thread

6.3 配置文件

在编译 RT-Thread SDK 前，请先选择并修改相关配置文件。

6.3.1 统一编译配置文件

RT-Thread SDK 编译配置：主要告诉编译脚本，RT-Thread 或 Bare-metal 的工程位置，以及配套工程的具体配置。RT-Thread SDK 默认配置位于

`<SDK>/device/rockchip/.chip/rockchip_rk3506_g_evb1_smp_defconfig`，相关配置如下：

```
RK_AMP=y                                # AMP RTOS / Bare-metal 支持
RK_AMP_FIT_ITS="smp.its"                # RT-Thread 固件打包配置文件
RK_UBOOT_CFG_FRAGMENTS="rk-amp"         # 增加 AMP U-Boot config 配置文件
RK_UBOOT_SPL=y                          # 使用 SPL
RK_USE_FIT_IMG=y                        # 使用 FIT 格式打包固件
# RK_KERNEL is not set
RK_PARAMETER="parameter-128M-smp.txt"   # 分区表配置文件
# root.img 的资源路径, rtos/bsp/rockchip/rk3506-32 的相对路径
RK_AMP_RTT_ROOT_DATA="../../applications/lvgl_demo/rk_demo/resource"
RK_UPDATE=y                             # 打包 update.img h
```

6.4 RT-Thread 固件打包配置文件

RT-Thread 固件打包使用标准的 FIT 格式。FIT 格式是 U-Boot 原生支持并且主推的。通过修改 ITS 配置文件，可以支持丰富的配置，RT-Thread 的 SDK 也通过脚本解析 ITS 配置文件，来完成工程的搭建，具体可以参考 **RT-Thread** 资源配置 章节，完整 ITS 文件如下：

```
/*
 * Copyright (C) 2024 Rockchip Electronic Co.,Ltd
 *
 * Simple U-boot fit source file containing ATF/OP-TEE/U-Boot/dtb/MCU
 */

/dts-v1/;

/ {
    description = "FIT Image with RT-thread";
    #address-cells = <1>;

    images {

        amp0 {
            description = "rtos-smp";
            data = /incbin/"rtt0.bin";
            type = "firmware";
            compression = "none";
            arch = "arm";
            os = "rtos";
            cpu = <0xf00>; // mpidr
            thumb = <0>; // 0: arm or thumb2; 1: thumb
        }
    }
}
```

```

hyp          = <0>;          // 0: e11/svc; 1: e12/hyp
load         = <0x00100000>;
compile {
    size      = <0x07f00000>;  // RAM 大小
    sys       = "rtt";         // 编译 RT-Thread 系统
    core      = "ap";          // 编译 AP 核
    rtt_config = "board/evb1/smp_defconfig"; // RT-Thread 配置文件
};
udelay       = <10000>; //delay uboot会延迟10000us进入系统
hash {
    algo      = "sha256";
};
};

configurations {
    default = "conf";
    conf {
        description = "RK3506-rtt-smp";
        loadables = "amp0";

        signature {
            algo = "sha256,rsa2048";
            padding = "pss";
            key-name-hint = "dev";
            sign-images = "loadables";
        };
    };
};
};
};

```

6.5 分区表配置文件

在开发过程中，需要根据实际使用的存储介质容量大小和 AMP 固件大小等调整分区表配置。

手动修改分区表配置文件 `parameter.txt`。分区表配置文件在：

`<AMP_SDK>/device/rockchip/.chip/$RK_PARAMETER`。添加分区格式为 `start@size(part_name)`，单位为 sector（512 Byte）。例如 `device/rockchip/rk3506/parameter-128M-smp.txt`：

```

FIRMWARE_VER:8.1
MACHINE_MODEL:RK3506
MACHINE_ID:007
MANUFACTURER: RK3506
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3506
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0
CMDLINE:mtdparts=:0x00002000@0x00002000 (uboot),0x00004000@0x00004000 (amp),0x00020000@0x00008000 (rootfs),-@0x00028000 (userdata:grow)
uuid:rootfs=614e0000-0000-4b53-8000-1d28000054a9

```

6.6 统一编译命令

RT-Thread SDK 统一编译命令如下，支持一键编译和打包等功能：

```
./build.sh chip           # 选择 SoC 平台
./build.sh lunch          # 选择默认配置文件
./build.sh                # 一键编译打包
./build.sh uboot          # 单独编译 U-Boot
./build.sh amp            # 单独编译 RT-Thread / Bare-metal
./build.sh cleanall       # 清除所有
./build.sh help           # 获取帮助
```

`./build.sh amp` 会读取配置文件中的 `RK_AMP_FIT_ITS` 指定的 ITS，根据 ITS 打包编译 RT-Thread，如 `rockchip_rk3506_g_evb1_smp_defconfig` 中的 `RK_AMP_FIT_ITS="smp.its"`

6.7 单独编译命令

6.7.1 U-Boot 编译命令

使用 SDK 下命令编译：

```
./build.sh uboot
```

进入到 U-Boot，U-Boot 作为独立组件单独编译时，针对 RK3506 平台，SDK 中对 `uboot` 有相关裁剪定制的配置，如下说明：

配置名称	说明
rk3506_defconfig	RK3506 基础配置
rk-amp.config	RK3506 AMP/SMP 子配置

使用方法：`./make.sh 基础配置+ 子配置`

参考命令如下：

```
cd <SDK>/u-boot/
./make.sh CROSS_COMPILE=<SDK>/prebuilts/gcc/linux-x86/arm/gcc-arm-10.3-2021.07-x86_64-arm-none-linux-gnueabi/bin/arm-none-linux-gnueabi- rk3506 rk-amp -- spl-new
```

6.7.2 RT-Thread 编译命令

使用 SDK 下命令编译：

```
./build.sh amp
```

在 SDK 下编译 RT-Thread 使用的配置文件需要在 `smp.its` 中指定，参考 **AMP 固件打包配置文件** 章节，SDK 中默认使用的配置文件为 `/rtos/bsp/rockchip/rk3506-32/board/evb1/smp_defconfig`，`smp.its` 配置如下：

```
rtt_config = "board/evb1/smp_defconfig"
```

进入 RTOS 目录下编译：

RT-Thread 作为独立组件单独编译时，使用的配置文件为 `.config`，`scons --menuconfig` 会读取 `.config` 中的内容，修改该完后保存会自动生成 `rtconfig.h` 文件，参考命令如下：

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/  
./build.sh 0  
./mkimage.sh
```

客户在使用时，可以在 `/rtos/bsp/rockchip/rk3506-32/` 使用 `scons --menuconfig` 修改配置，再将 `board/evb1/smp_defconfig` 替换成 `.config`，这样便可直接在 SDK 下使用 `./build.sh amp` 生成修改配置后的固件，参考操作如下：

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/  
scons --menuconfig  
# 保存修改  
cp .config board/evb1/smp_defconfig  
cd <SDK>/  
./build.sh amp
```

结合 SDK 统一编译方式，可以按照如下步骤配置编译 RT-Thread：

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32          # 进入 RT-Thread 工程  
cp board/evb1/smp_defconfig .config          # 基于 SDK 提供的默认配置  
scons --menuconfig                            # menuconfig 配置工程  
# 配置完成后保存在.config中，会自动生成rtconfig.h  
cp .config board/evb1/smp_defconfig          # 可以直接覆盖 SDK 的默认配置，回到SDK根目  
录下编译  
cd <SDK>/  
./build.sh
```

6.7.3 root.img 打包

RT-Thread 支持 FAT 文件系统，具体文件系统细节可以参考 《**Rockchip_Developer_Guide_RT-Thread_SPIFLASH_CN.pdf**》。

使用 SDK 下命令编译：

```
# 编译 RT-Thread 时会打包 root.img  
./build.sh amp
```

如果 `<SDK>/device/rockchip/.chip/` 有指定 `RK_AMP_RTT_ROOT_DATA` 的路径，运行 `./build.sh` 会在 `<SDK>/rockdev` 下生成对应的 `root.img`

```
# 这是一个基于<SDK>/rtos/bsp/rockchip/rk3506-32 的相对路径  
RK_AMP_RTT_ROOT_DATA="../../applications/lvgl_demo/rk_demo/resource"
```

除了 SDK 下 `./build.sh amp` 打包 `root.img` 外，还可以使用脚本打包：

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/
# 方式一：使用 ./mkroot.sh 脚本
# ./mkroot.sh root <resource_file> <parameter_file> <page_size_bytes>
<target_file>
# 如：nand flash page 2048 bytes, 打包 resource/root 生成 Image/root.img
./mkroot.sh root resource/root board/evb1/parameter-128M-smp.txt 2048
Image/root.img

# 方式二：使用 ./build.sh 脚本
# ./build.sh root <resource_file> <PAGE_SIZE(bytes)>
# 如：nand page 2048 bytes, 打包 resource/root 生成 Image/root.img
./build.sh root resource/root 2048

# root 使用的 mini ftl 是只读的，具体可以参考 《Rockchip_Developer_Guide_RT-Thread_SPIFLASH_CN.pdf》
```

使用文件系统参考如下步骤：

1. `scons --menuconfig` 开启文件系统支持：

```
# SPI NAND Flash 支持
RT_USING_SNOR=n                # 关闭 SPI Nor 驱动，通常外设仅有一种 SPI Flash
RT_USING_MTD_NAND=y
RT_USING_SPINAND=y
RT_SPINAND_SPEED=80000000      # IO 接口速率
RT_USING_SPINAND_FSPI_HOST=y   # FSPI 控制器方案
RT_USING_FTL=y                 # 启用 SPI Nand FTL 存储管理算法
RT_USING_DHARA=y               # 启用 SPI Nand DHARA FTL 存储管理算法（与 Mini FTL 兼容）
RT_USING_DFS_MNTTABLE=y        # 启动 DFS mnt table 自动挂载支持
RT_USING_DFS_ELMFAT=y          # 启用 elm FatFs
RT_DFS_ELM_MAX_SECTOR_SIZE=2048 # 设置 elm FatFs sector size, 设定为 flash pagesize, 通常为 2KB/4KB, 参考颗粒手册

# RT-Thread DFS 支持
CONFIG_RT_USING_DFS=y
CONFIG_DFS_USING_POSIX=y
CONFIG_DFS_USING_WORKDIR=y
CONFIG_DFS_FILESYSTEMS_MAX=4    # 可以挂载 (mount) 的文件系统数量
CONFIG_DFS_FILESYSTEM_TYPES_MAX=4 # 可以挂载 (mount) 的文件系统数量
CONFIG_DFS_FD_MAX=16
CONFIG_RT_USING_DFS_MNTTABLE=y  # 开机自动挂载文件系统，参考 mnt.c
CONFIG_RT_USING_DFS_ELMFAT=y    # FAT 文件系统支持

# elm-chan's FatFs, Generic FAT Filesystem Module
CONFIG_RT_DFS_ELM_CODE_PAGE=437
CONFIG_RT_DFS_ELM_WORD_ACCESS=y
CONFIG_RT_DFS_ELM_USE_LFN_3=y
CONFIG_RT_DFS_ELM_USE_LFN=3
CONFIG_RT_DFS_ELM_LFN_UNICODE_0=y
CONFIG_RT_DFS_ELM_LFN_UNICODE=0 # FAT 文件系统 UNICODE 编码类型
CONFIG_RT_DFS_ELM_MAX_LFN=255
```

```
CONFIG_RT_DFS_ELM_DRIVES=3           # 允许挂载的 FAT 文件系统数量
CONFIG_RT_DFS_ELM_MAX_SECTOR_SIZE=2048
CONFIG_RT_DFS_ELM_REENTRANT=y
CONFIG_RT_DFS_ELM_MUTEX_TIMEOUT=3000
CONFIG_RT_USING_DFS_DEVFS=y

CONFIG_RT_USING_USER_DATA_PART=y      # 没有预烧录 userdata.img 可以开启，开机自动
格式 userdata 分区并且挂载在 /data 下
```

2. 打包 root.img
3. 烧录固件，成功挂载可以使用 `ls` 查看

```
msh />ls
Directory /:
data          <DIR>
udisk         <DIR>
sdcard        <DIR>
demo.txt      24
```

6.7.4 RK HAL 编译命令

RK HAL 作为独立组件单独编译时，参考命令如下：

```
cd <AMP_SDK>/hal/project/rk3506/GCC
./build.sh <cpu_id 0~3 or all>
cd ..
./mkimage.sh

make -j8
make clean
```

7. RT-Thread 资源配置

7.1 RT-Thread 配置文件

通常来说，芯片级别的工程中，会预设好几个板级配置，使用工程中，通过 `scons --menuconfig` 指令修改 `CONFIG_RT_BOARD_NAME` 配置，达到同一个工程适配多个板子的需求。

因此，RT-Thread 配置的内容包括：

```

<SDK>/rtos/bsp/rockchip/rk3506-32/
├── applications
├── board
│   ├── common                # 通用配置
│   ├── Kconfig
│   ├── evb1                  # 板级配置
│   └── SConscript
├── build.sh                  # 编译脚本，内含一些编译配置
└── ...

```

- 通用配置部分：芯片的基础配置，为工程必选代码，为所有板级工程服务。
- 板级配置部分：板级配置，为具体板子配置相关功能，比如 GPIO、UART、I2C 等。
- 编译命令：编译命令可以实现编译时的系统传参，提供灵活的编译指令。有 `build.sh` 脚本时，参数可以直接定义在 `build.sh` 中。或者 `export` 特定编译参数后，使用 `scons` 命令编译。

7.1.1 板级相关文件

RT-Thread 工程资源配置，包括通用配置，以及板级配置。

通用配置文件包括：

```

# 通用配置
<SDK>/rtos/bsp/rockchip/rk3506-32/board/common/board_base.c
<SDK>/rtos/bsp/rockchip/rk3506-32/board/common/iomux_base.c

```

通用配置，定义了硬件启动的初始化顺序和默认硬件资源同时提供大量的 `RT_WEAK` 定义，方便用户在板级配置中进行对应的替换。

```

RT_WEAK const struct clk_init clk_inits[];           // 弱定义结构体，可以在板级配置
中重定义
RT_WEAK void rt_hw_iomux_config(void);               // 弱定义函数，可以在板级配置中
重定义
// ...                                               // 不同芯片的弱定义资源不一致，
不一一列举

void rt_hw_board_init(void)
{
    // ... 初始化顺序，不要修改
}

```

板级配置文件包括：

```

# 板级配置
<AMP_SDK>/rtos/bsp/rockchip/rk3506-32/board/evb1/board.c
<AMP_SDK>/rtos/bsp/rockchip/rk3506-32/board/evb1/iomux.c

```

板级配置中，对需要的 `RT_WEAK` 资源进行重定义。


```
// <AMP_SDK>/rtos/bsp/rockchip/rk3562-32/board/rk3562_evbl_lp4x/iomux.c

void rt_hw_iomux_config(void)
{
    // ...
}
```

7.1.2 编译相关文件

RT-Thread 使用 `scons` 编译，编译涉及的文件有：

```
cd <SDK>/rtos/bsp/rockchip/rk3506-32/

rk3506-32/rtconfig.h          # 编译使用的配置文件
rtconfig.py                  # 编译脚本，决定编译链位置，编译命令的传入参数等
SConscript                   # SCONS 链接文件
SConstruct                   # SCONS 链接文件

build.sh                     # RT-Thread AP Core 快捷编译脚本
```

7.2 RT-Thread 内存资源

RT-Thread 的内存分配，由编译的链接文件分配。

```
<SDK>/rtos/bsp/rockchip/rk3506-32/gcc_arm.ld.S
```

7.2.1 AP 运行内存配置

```
/* cat <SDK>/rtos/bsp/rockchip/rk3506-32/gcc_arm.ld.S */

MEMORY
{
    SRAM    (rxw) : ORIGIN = 0xffff80000, LENGTH = 64K          /* SYSTEM SRAM */
    DRAM    (rxw) : ORIGIN = FIRMWARE_BASE, LENGTH = DRAM_SIZE /* DRAM */
#ifdef RT_USING_SMP
    LINUX_RPMSG (rxw) : ORIGIN = LINUX_RPMSG_BASE, LENGTH = LINUX_RPMSG_SIZE
#endif
}

# cat <SDK>/rtos/bsp/rockchip/rk3506-32/rtconfig.py
# 做变量转换，连接 build.sh 脚本和 gcc_arm.ld.S 文件
CFLAGS += ' -DFIRMWARE_BASE={a} -DDRAM_SIZE={b} -DLINUX_RPMSG_BASE={c} -
DLINUX_RPMSG_SIZE={d} '.format(a=PRMEM_BASE, b=PRMEM_SIZE, c=LINUX_RPMSG_BASE,
d=LINUX_RPMSG_SIZE)

/* cat <SDK>/rtos/bsp/rockchip/rk3506-32/build.sh */

# TODO: Please plan the memory according to the actual needs of the project.
```

```
# Requiring 1M alignment.
CPU0_MEM_BASE=0x03900000
CPU1_MEM_BASE=0x03a00000
CPU2_MEM_BASE=0x03e00000
CPU0_MEM_SIZE=0x00100000
CPU1_MEM_SIZE=0x00100000
CPU2_MEM_SIZE=0x00100000
```

RT-Thread 的内存配置，处于 `gcc_arm.ld.S` 中，为了方便编译配置，部分参数，通过 `rtconfig.py` 转化到 `build.sh` 中，方便编译修改。

AP Core 中配置的地址信息，即使真实 DDR 的物理地址信息。

DRAM 可以通过 `build.sh` 中的参数进行配置：

```
CPU0_MEM_BASE=0x03900000
CPU0_MEM_SIZE=0x00100000
export RTT_PRMEM_BASE=$(eval echo \${CPU$1_MEM_BASE})          /* DRAM 起始位置
*/
export RTT_PRMEM_SIZE=$(eval echo \${CPU$1_MEM_SIZE})           /* DRAM 容量大小
*/
```

以上变量，对应 `gcc_arm.ld.S` 的 DRAM 区域：

```
DRAM (rxw) : ORIGIN = FIRMWARE_BASE, LENGTH = DRAM_SIZE        /* DRAM */
```

变量名的不同，通过 `rtconfig.py` 转化。

注意：对于在 SDK 使用 `./build.sh` 命令编译的固件，DRAM 大小是在 `smp.its` 中定义，具体可以直接参考 `smp.its`，除了前 1M 空间被系统使用外，其余 RAM 都可以给 RT-Thread 使用，配置如下：

```
# load0 为 DRAM 的起始地址，size 为 DRAM 大小，分别对应 ./build.sh 中的 CPU0_MEM_BASE 和
CPU0_MEM_SIZE，smp.its 中定义 DRAM 起始地址 1M，大小 127M

amp0 {
    description = "rtos-smp";
    data        = /incbin/"rtt0.bin";
    type        = "firmware";
    compression = "none";
    arch        = "arm";
    os          = "rtos";
    cpu         = <0xf00>; // mpidr
    thumb       = <0>;      // 0: arm or thumb2; 1: thumb
    hyp         = <0>;      // 0: el1/svc; 1: el2/hyp
    load        = <0x00100000>;
    compile {
        size    = <0x07f00000>;
        sys     = "rtt";
        core    = "ap";
        rtt_config = "board/evb1/smp_defconfig";
    };
    udelay      = <10000>;
    hash {
        algo    = "sha256";
    };
};
```

```
};  
};
```

7.3 编译链接脚本介绍

7.3.1 概述

RK3506 使用 ARM 编译链接脚本来管理 CPU 内部的内存分配。关于编译链接脚本的详细信息和使用方法，用户可参考 ARM 官方相关文档以及链接脚本文件。代码编译完成后，会根据编译链接脚本产生对应的代码及数据内存分配段。参考如下：

链接脚本相关数据段说明：

名称	类型	地址	说明
text	代码段	0	存放代码段和其他ro的数据
zero.table	清0表	紧接text	存放清0段的起始和结束地址
data	数据段	紧接zero.table	存放rw数据段
ttb	mmu表	紧接data（16K对齐）	mmu表，目前只配置了一级页表
share_mem	共享内存	紧邻ttb	存放共享数据
linux_share_mem	linux 共享内存	紧邻share_mem	存放linux共享数据
bss	bss段	紧邻linux_share_mem	清0数据段
heap	堆	紧邻bss段	堆，剩余内存全部分配给堆
stack	栈	紧邻heap	固定1KB大小，放在amp最后面

7.3.1.1 栈大小分配

在连接脚本文件gcc_arm.ld.S中，对于不同模式，堆栈大小都有独立的配置，用户可以根据需求进行修改。参考及说明如下：

```
__STACK_SIZE = 0x400;           # 不同模式下堆栈是独立的，所以配置也独立  
__FIQ_STACK_SIZE = 0x400;       # 主堆栈大小，即我们正常跑的模式下堆栈大小  
__IRQ_STACK_SIZE = 0x400;       # FIQ模式下堆栈大小  
__SVC_STACK_SIZE = 0x1000;      # IRQ模式下堆栈大小  
__ABT_STACK_SIZE = 0x400;       # SVC模式下堆栈大小  
__UND_STACK_SIZE = 0x400;       # ABORT异常下堆栈大小  
__TTB_SIZE = 0x4000;            # undefined异常下堆栈大小  
                                # MMU页表大小，请不要改
```

7.3.1.2 堆大小分配

堆的大小是根据实际用户使用情况动态分配的。它取决于栈的大小的分配，以及代码使用空间的情况。堆的大小计算方式可参考如下：

堆大小 = 内存总大小 - 栈大小 - 用户代码使用的空间

在连接脚本文件gcc_arm.ld.S中，堆的分配定义参考如下：

```
SECTIONS
{
    # .....
    . = ALIGN(16);
    __ALL_STACK_SIZE = __STACK_SIZE + __FIQ_STACK_SIZE + __IRQ_STACK_SIZE +
__SVC_STACK_SIZE + __ABT_STACK_SIZE + __UND_STACK_SIZE;
    __STACK_START = ORIGIN(DRAM) + LENGTH(DRAM) - __ALL_STACK_SIZE;
    __HEAP_SIZE = (__STACK_START - .);
    .heap :
    {
        PROVIDE(__heap_start = .);
        . += __HEAP_SIZE;           # 堆的介于 bss 和 stack 之间，其大小为系统扣除
所有开销之后所剩余的所有空间
        PROVIDE(__heap_end = .);
    } > DRAM

    .stack __STACK_START :
    {
        # .....
    } > DRAM
}
```

8. OTA 升级

OTA（Over-the-Air）技术是指通过无线通信的方式，远程升级设备固件或软件，而不需要通过传统的线缆连接进行手动更新。设备制造商可以通过 OTA 技术向大量分布在全球的设备推送最新的固件和软件更新，修复安全漏洞、提供新功能或提升设备性能。

RK3506 中采用 A/B 固件的方式进行 OTA 升级，也就是 A 固件在运行时，可以对 B 固件进行升级，升级成功后切换到 B 固件运行，它具有以下特点：

- 在OTA期间可以继续使用设备
- 固件升级失败时，可以回退到先前的固件继续工作

8.1 固件格式

有以下特点：

1. `update.img` 为有效固件，最终升级为分区分立升级。
2. 支持 GPT partition。

8.2 配置说明:

8.2.1 配置 parameter.txt

update.img 固件分区配置信息叫 parameter.txt，内容如下：

```
FIRMWARE_VER:8.1
MACHINE_MODEL:RK3506
MACHINE_ID:007
MANUFACTURER: RK3506
MAGIC: 0x5041524B
ATAG: 0x00200800
MACHINE: 3506
CHECK_MASK: 0x80
PWR_HLD: 0,0,A,0,1
TYPE: GPT
GROW_ALIGN: 0

CMDLINE:mtddparts=:0x00000600@0x00000000(loader),0x00002000@0x00002000(uboot),0x00
000800@0x00004000(misc),0x00002000@0x00004800(amp_a),0x00002000@0x00006800(amp_b)
,0x00002000@0x00008800(root),-@0x00028800(userdata:grow)
uuid:rootfs=614e0000-0000-4b53-8000-1d28000054a9

# 以0x00002000@0x00004800(amp_a)为例说明
# 其格式是 PartSize@PartOffset(Name)，PartSize与PartOffset大小单位为sector，也就是512
字节
# 每个分区信息之间通过逗号分隔，下一个分区的PartOffset为前一个分区的PartOffset+PartSize
# 最后一个分区定义时不用指定PartSize，软件会根据flash大小自动计算
# GPT 分区表存储在 NAND FLASH 的第一个block，所以 0x00000600@0x00000000(loader) 包含
gpt 表和 loader 固件，升级时会偏移跳过 gpt
# root 也可以进行 A/B 分区，客户可以根据需求自行修改
```

固件打包配置文件叫 package-file，里面指定了要打包那些固件到 update.img 中，在 SDK 中会根据使用的 parameter.txt 来生成 package-file。

注意：

- userdata分区需要放在最后一个。
- OTA 目前无法支持 gpt 分区表的升级，注意预留足够的分区空间
- 对于 spi nand，分区需要预留额外的 block 以防止出现坏块，小分区多留一个 block（256KB），大的分区多留 1MB。

8.2.2 UBoot 配置

使用 A/B 分区 UBoot 需要开启 A/B 分区支持：

```
CONFIG_ANDROID_AB=y
CONFIG_AVB_LIBAVB=y
CONFIG_AVB_LIBAVB_AB=y
CONFIG_AVB_LIBAVB_ATX=y
CONFIG_AVB_LIBAVB_USER=y
CONFIG_RK_AVB_LIBAVB_USER=y
```

8.2.3 RT-Thread 配置

```
CONFIG_RT_USING_NEW_OTA=y           # 使能OTA功能
CONFIG_RT_USING_NEW_OTA_MODE_AB=y   # 选择 A/B 分区
CONFIG_RT_USING_AVB_LIBAVB_AB=y     # LIBAVB 格式的 misc 分区
CONFIG_RT_USING_FWANALYSIS=y
```

可以开启文件系统和 USB 的支持，将升级包 `update.img` 放在文件系统中，文件系统参考上文 `root.img` 打包 章节，USB 详细说明和配置参考文档《[Rockchip_Developer_Guide_RT-Thread_USB_CN.pdf](#)》

USB 配置如下：

```
# Using USB
CONFIG_RT_USING_USB=y
CONFIG_RT_USING_USB_HOST=y
CONFIG_RT_USBH_MSTORAGE=y
CONFIG_UDISK_MOUNTPOINT="udisk"
CONFIG_RT_USBD_THREAD_STACK_SZ=4096

CONFIG_RT_USING_DWC2_USBH=y          # OTG0
CONFIG_RT_USING_DWC2_USBH1=y        # OTG1
```

8.2.4 版本号修改

OTA程序默认对比当前运行固件和待升级 `update.img` 的版本号来决定是否要升级，如果版本号不一致，则会进行升级，否则不进行升级。这部分代码在：`rk_ota_fw_version_chk`，客户可以自己修改逻辑。

修改版本号方法如下，修改板级 `board` 目录下的 `parameter.txt` 文件，例如 `board/evb/parameter.txt`

```
FIRMWARE_VER: 1.0.0
```

版本号没有更新导致不会触发升级
类似如下log提示：

```
OTA: current fw_version 0x1000000 new fw_version 0x1000000
OTA: rk_ota_fw_version_chk: fw version is same
OTA: rk_ota_process: no need update
```

8.3 A/B固件启动信息

A/B固件启动信息存放在 `misc` 分区中，具体内容如下：

```
typedef struct fw_ab_slot_data_t
{
    /* Slot priority. Valid values range from 0 to AVB_AB_MAX_PRIORITY,
     * both inclusive with 1 being the lowest and AVB_AB_MAX_PRIORITY
     * being the highest. The special value 0 is used to indicate the
     * slot is unbootable.
     */
    uint8_t priority;

    /* Number of times left attempting to boot this slot ranging from 0
     * to AVB_AB_MAX_TRIES_REMAINING.
     */
    uint8_t tries_remaining;

    /* Non-zero if this slot has booted successfully, 0 otherwise. */
    uint8_t successful_boot;

    /* Reserved for future use. */
    uint8_t reserved[1];
} fw_ab_slot_data;
```

使用PC工具升级后，第一次启动时，UBoot 程序会初始化 A/B 固件启动信息，见如下打印信息：

```
A/B-slot: _a, successful: 0, tries-remain: 7
```

第一次启动时 A 固件的 `priority` 高，所以表示从 A 固件启动，`tries_remaining` 表示对应固件剩余启动尝试次数，`successful_boot` 表示对应固件是否启动成功，设置为1后下次重启则会继续从这个固件启动，A 固件启动成功后的打印信息如下：

```
A/B-slot: _a, successful: 1, tries-remain: 0
```

`successful_boot` 标记需要 `rt-thread` 启动后由应用程序去设置，SDK中默认设置启动成功的代码在：
`bsp/rockchip/rk3506-32/applications/main.c`

```
// 这里建议将其移到一个能够表示业务成功跑起来的地方。
int main(void)
{
    ...
#ifdef RT_USING_NEW_OTA
    rk_ota_set_boot_success();
#endif
    ...
}
```

可通过调用函数 `int fw_slot_get_current_running(uint32_t *cur_slot)` 来判断当前运行的固件是A还是B。`cur_slot=0` 表示是A固件，`cur_slot=1` 表示是B固件。

8.4 OTA触发方式

OTA代码在：`components/ota/` 目录，其中 `ota_new.c` 中 `rk_ota_process` 为主程序。

8.4.1 默认触发方式

开机后会在以下目录查找 `update.img` 进行升级尝试，其他路径客户可以根据需求自行修改。

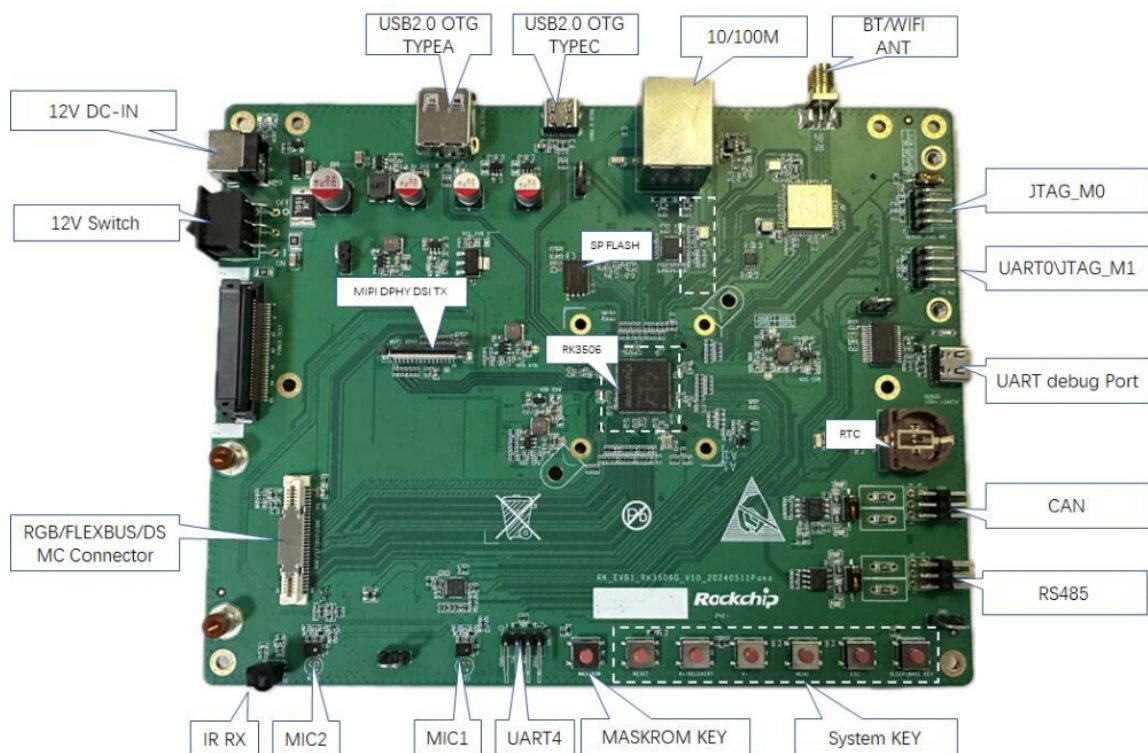
```
RT_WEAK const char *const ota_fw_path[] =
{
    "/",
    "/udisk",
    "/sdcard"
};
```

8.4.2 其它触发方式

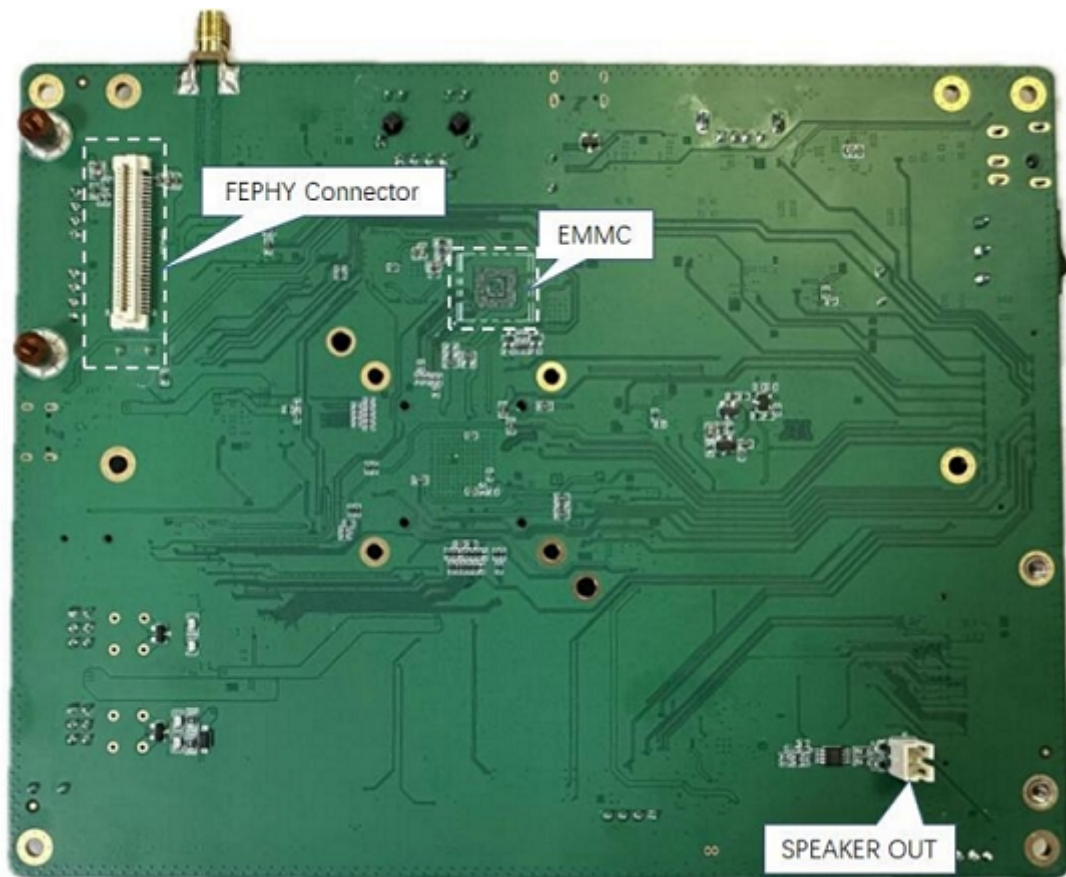
例如网络下载。网络下载部分目前需要客户自己实现，然后调用 `rk_ota_process` 触发升级

9. 刷机说明

RK3506 EVB 1开发板正面接口分布图如下：



RK3506 EVB1 开发板背面接口分布图如下：



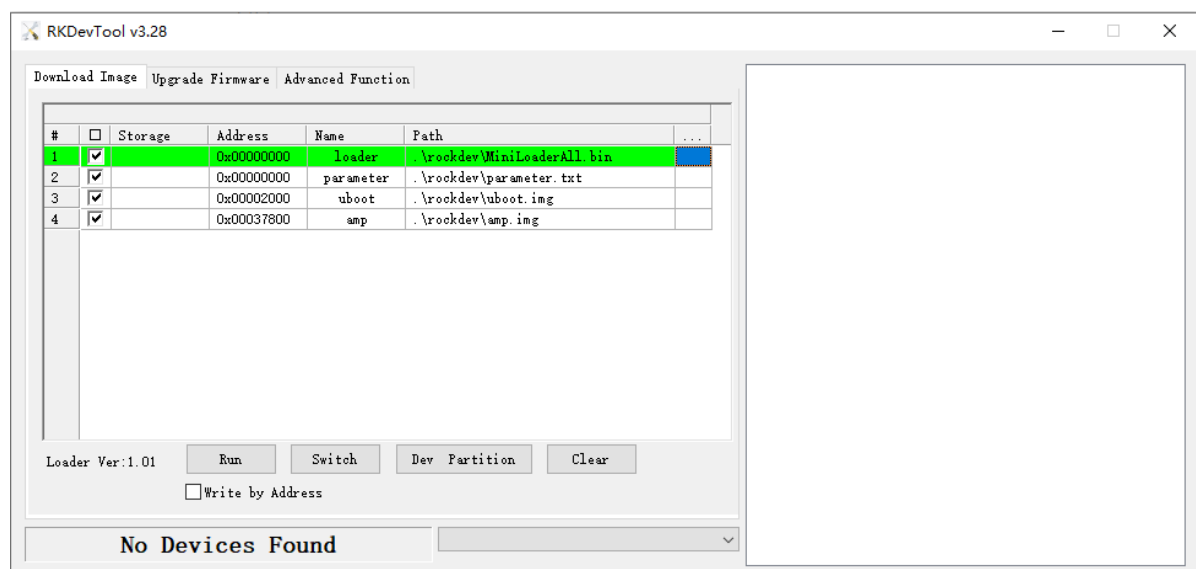
9.1 Windows 刷机说明

SDK 提供 Windows 烧写工具(工具版本需要 V3.28 或以上), 工具位于工程根目录:

```
tools/
└─ windows/RKDevTool
```

如下图, 编译生成相应的固件后, 设备烧写需要进入 MASKROM 或 BootROM 烧写模式, 连接好 USB 下载线后, 按住按键“MASKROM”不放并按下复位键“RST”后松手, 就能进入MASKROM 模式, 加载编译生成固件的相应路径后, 点击“执行”进行烧写, 也可以按 “recovery”按键不放并按下复位键 “RST” 后松手进入 loader 模式进行烧写, 下面是 MASKROM 模式的分区偏移及烧写文件。

(注意: Windows PC 需要在管理员权限运行工具才可执行)



注：烧写前，需安装最新 USB 驱动，驱动详见：

```
<SDK>/tools/windows/DriverAssitant_v5.13.zip
```

9.2 Linux 刷机说明

Linux 下的烧写工具位于 tools/linux 目录下(Linux_Upgrade_Tool 工具版本需要 V2.36 或以上)，请确认你的板子连接到 MASKROM/loader rockusb。比如编译生成的固件在 rockdev 目录下，升级命令如下：

```
sudo ./upgrade_tool ul rockdev/MiniLoaderAll.bin -noreset
sudo ./upgrade_tool di -p rockdev/parameter.txt
sudo ./upgrade_tool di -u rockdev/uboot.img
sudo ./upgrade_tool di -amp rockdev/amp.img
sudo ./upgrade_tool rd
```

或升级打包后的完整固件：

```
sudo ./upgrade_tool uf rockdev/update.img
```

或在根目录，机器在 MASKROM 状态运行如下升级：

```
./rkflash.sh
```

10. 运行调试

10.1 系统启动

系统启动方式有以下几种：

1. 固件升级后，自动重新启动；
2. 插入USB供电直接启动；
3. 有电池供电的设备，按Reset键启动；

注：不同的硬件设计，其上电启动方式也会有不同的区别。

10.2 串口调试

RK3506 RT-Thread SDK中主核默认用UART0打印。

串口通信配置信息如下：

波特率：1500000

数据位：8

停止位：1

奇偶校验: none

流控: none

系统启动后, 能够看到如下u-boot打印:

```
AMP: Brought up primary cpu[f00, self] with state 0x10, entry 0x03900000 ...OK
```

进入RT-Thread 系统后能看到系统信息的打印:

```
\ | /
- RT -      Thread Operating System
/ | \      4.1.1 build Oct 21 2024 16:17:13
2006 - 2022 Copyright by RT-Thread team
Hi, this is RT-Thread!!
msh >I/TC: Secondary CPU 1 initializing
*I/TC: Secondary CPU 2 initializing
*I/TC: Secondary CPU 1 switching to normal world boot
*I/TC: Secondary CPU 2 switching to normal world boot
```